

CYBER SECURITY TASK 3

API Security Risk Analysis

Read-Only Security Assessment of a Public Test API

Submitted by
Ann Maria Sony

Internship: Cyber Security **Organization:** Future Interns

Tool: Postman

Target: JSONPlaceholder Test API

Assessment Type: Read-Only API Security Assessment

Assessment Scope: Public/demo API endpoints only

No exploitation • No DoS testing • No production systems

2026

1. Executive Summary

Application Programming Interfaces (APIs) are a critical component of modern software systems. They allow applications, services, and users to exchange data and perform operations programmatically. Because APIs frequently expose application data and functionality, improper authentication, authorization, input validation, and data handling can introduce security risks.

As part of Future Interns Cyber Security Task 3, a read-only API security assessment was performed against JSONPlaceholder, a publicly available test API designed for learning and development.

The assessment was conducted using Postman. The tested endpoints included GET /posts, GET /posts/1, GET /users, and GET /users?id=1. The assessment focused on observable authentication requirements, HTTP response codes, response headers, query handling, and the information returned by the API.

The selected endpoints returned HTTP 200 OK without authentication. Because JSONPlaceholder is intentionally designed as a public testing API and uses synthetic data, this behavior is not treated as a confirmed vulnerability. Instead, it is documented as a security consideration that would require stronger controls if the same design were used for private production data.

Overall, the assessment demonstrates the practical workflow of API security review: observe behavior, assess security implications in context, classify risk conservatively, and recommend remediation without performing unauthorized exploitation.

2. Scope, Objectives & Ethical Considerations

2.1 Assessment Scope

In scope: public JSONPlaceholder endpoints, GET requests, HTTP response codes, response headers, authentication requirements, response data exposure, and basic API security observations.

Out of scope: production/private APIs, exploitation, authentication bypass attempts, denial-of-service or flooding tests, modification/deletion of data, and access to unauthorized private information.

2.2 Objectives

- Analyze a public/demo API.
- Identify common API security risks and security considerations.
- Examine authentication and access-control requirements.
- Inspect HTTP headers and API responses.
- Assess the type and amount of information exposed by endpoints.
- Classify observations according to severity.
- Explain potential business impact.
- Recommend practical security improvements.

2.3 Ethical Considerations

The assessment followed a read-only and non-destructive methodology. Only a public test API intended for learning was accessed. No attempts were made to bypass authentication, exploit vulnerabilities, overload the service, modify data, or access private systems.

Important: JSONPlaceholder is specifically designed as a public test API. Therefore, observations such as unauthenticated access are documented as security considerations rather than confirmed vulnerabilities.

3. API Tested & Testing Environment

3.1 Target API

The API selected for this assessment was JSONPlaceholder, a free public REST API designed for testing and development purposes.

Base URL: <https://jsonplaceholder.typicode.com>

3.2 Testing Tool

Postman was used to send HTTP GET requests, verify status codes, inspect authentication settings, review request/response headers, examine JSON response data, and capture evidence.

3.3 Endpoints Tested

No.	Method	Endpoint	Purpose
1	GET	/posts	Public access and response
2	GET	/posts/1	Direct object retrieval
3	GET	/users	User-data exposure
4	GET	/users?id=1	Targeted user retrieval/query handling
5	GET	/posts — response headers	Header/security configuration review

3.4 Testing Environment

Tool: Postman **Protocol:** HTTPS **Request type:** GET **Mode:** Read-only **Authentication supplied:** None **Data:** Synthetic/demo JSON

3.5 Security Areas Considered

Authentication, authorization/object-level access, data exposure, HTTP security headers/CORS, and normal query-parameter handling were considered. Where the available evidence was insufficient, the report explicitly states that the issue was not confirmed or not assessed.

4. Methodology & Testing Procedure

Step 1 — Select the API

JSONPlaceholder was selected as an approved public/demo API for safe learning and testing.

Step 2 — Review API Documentation

Available resources and GET routes were reviewed to understand expected endpoint behavior before testing.

Step 3 — Test Endpoints

Normal GET requests were sent through Postman to the selected Posts and Users endpoints.

Step 4 — Inspect Authentication, Headers & Response Data

Authentication settings, HTTP status codes, response headers, and JSON response fields were reviewed.

Step 5 — Identify Security Risks

Observed behavior was compared against common API security concerns such as unauthenticated access and excessive data exposure.

Step 6 — Classify Risk Severity

Observations were classified conservatively using context, impact, and the fact that the target is intentionally public.

Step 7 — Suggest Remediation

Practical controls were recommended for a production SaaS API.

Step 8 — Document Findings

The methodology, evidence, findings, severity, impact, recommendations, and conclusion were documented.

Testing principle: Identify risk, don't exploit it. The assessment deliberately avoided destructive testing, authentication bypass, injection attacks, denial-of-service testing, and unauthorized access.

5.1 Test 1 — Public /posts Endpoint

Request: GET https://jsonplaceholder.typicode.com/posts

Purpose: The purpose was to determine whether the endpoint could be accessed without an authentication token or API key.

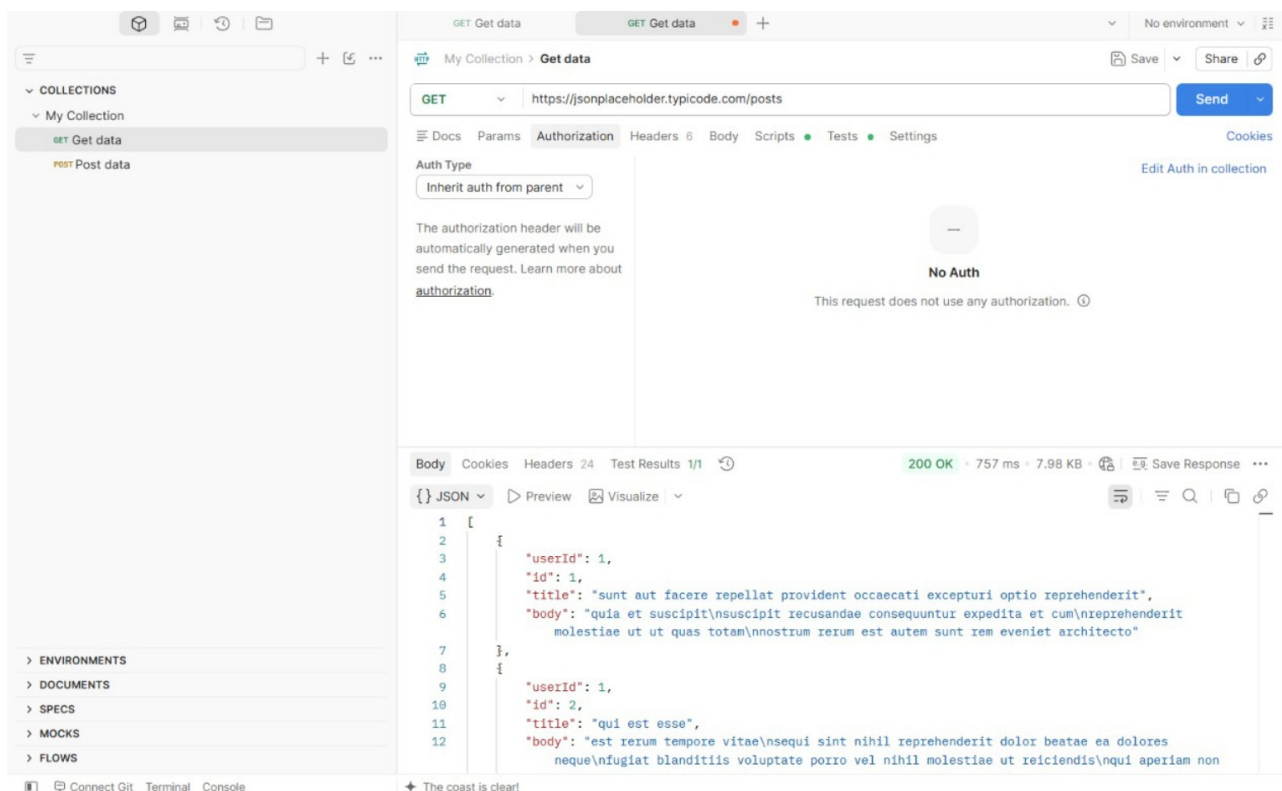
Observed Result: HTTP 200 OK was returned while Postman showed No Auth. The endpoint is publicly accessible.

Security Analysis: In a private production API, unrestricted access to protected resources could be an authentication/access-control risk. For JSONPlaceholder, public access is expected.

Risk Classification: Informational / Low

Recommended Control: Require authentication for protected endpoints and enforce server-side authorization.

Evidence 1 — GET /posts: public endpoint, No Auth, HTTP 200 OK



What the evidence demonstrates: The screenshot supports the observed request/response behavior described above. It is not, by itself, proof of an exploitable vulnerability.

5.2 Test 2 — HTTP Response Header Analysis

Request: GET https://jsonplaceholder.typicode.com/posts

Purpose: The response headers were inspected for security-relevant configuration, information disclosure, and CORS-related settings.

Observed Result: Headers observed included Content-Type: application/json, Cache-Control, Access-Control-Allow-Credentials: true, Content-Encoding, ETag, and Expires.

Security Analysis: The credentialed CORS header requires contextual review, but the header alone does not establish a CORS vulnerability.

Risk Classification: Informational / Low

Recommended Control: Restrict allowed origins, enable credentials only when required, and review CORS configuration.

Evidence 2 — Response headers inspected for GET /posts

The screenshot displays a REST client interface with the following details:

- Request:** GET https://jsonplaceholder.typicode.com/posts
- Response Status:** 200 OK
- Response Headers:**

Key	Value	Description
:status	200	
date	Fri, 14 Aug 2026 16:30:14 GMT	
content-type	application/json; charset=utf-8	
access-control-allow-credentials	true	
cache-control	max-age=43200	
content-encoding	gzip	
etag	W/"6b80-Ybsq/K6GwwqrYkAsFxqDXGC7DoM"	
expires	-1	
nel	{"report_to": "theroku-nel", "response_headers": [{"Via": 1, "max_age": 360...	

What the evidence demonstrates: The screenshot supports the observed request/response behavior described above. It is not, by itself, proof of an exploitable vulnerability.

5.3 Test 3 — /posts/1 Object Access

Request: GET https://jsonplaceholder.typicode.com/posts/1

Purpose: The test examined direct access to an individual resource and whether authentication was required.

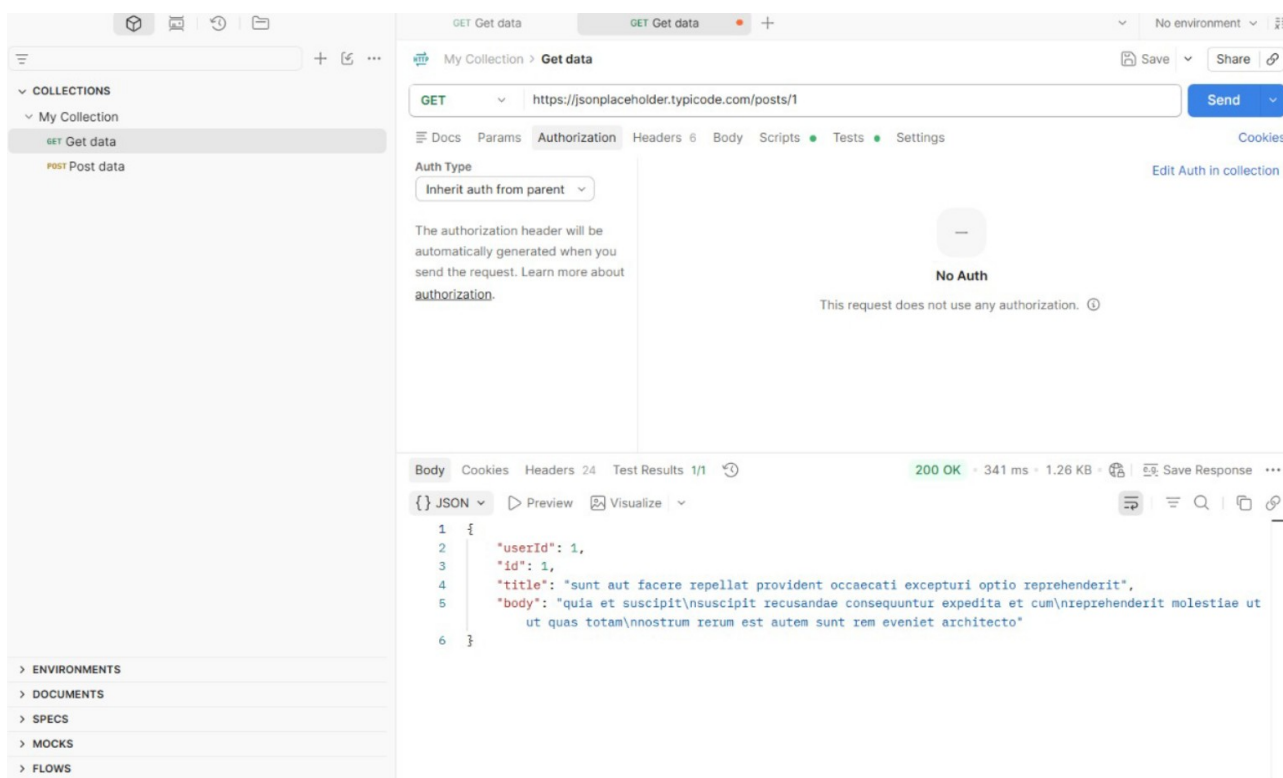
Observed Result: HTTP 200 OK was returned with a single post object while No Auth was used.

Security Analysis: In a private production API, insufficient object-level authorization could lead to BOLA. This assessment did not attempt to prove or exploit BOLA.

Risk Classification: Informational / Low

Recommended Control: Require authentication for protected resources and perform server-side object-level authorization checks.

Evidence 3 — GET /posts/1: direct object retrieval, No Auth, HTTP 200 OK



What the evidence demonstrates: The screenshot supports the observed request/response behavior described above. It is not, by itself, proof of an exploitable vulnerability.

5.4 Test 4 — /users Endpoint

Request: GET https://jsonplaceholder.typicode.com/users

Purpose: The test examined the type and amount of user-related information returned by the API.

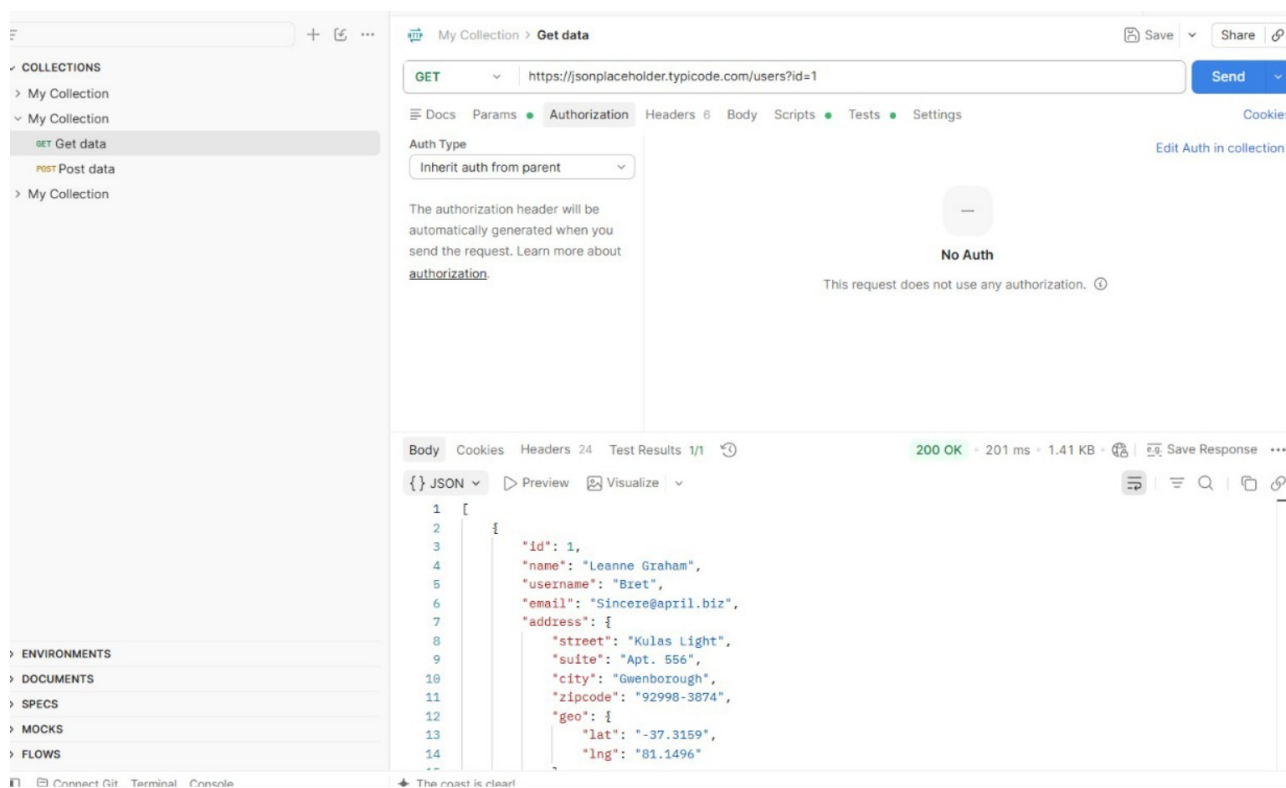
Observed Result: HTTP 200 OK returned multiple user records with fields such as ID, name, username, email, address, and geographic coordinates.

Security Analysis: Returning unnecessary personal/location fields could create privacy and data-exposure risks in a real service. JSONPlaceholder uses synthetic data.

Risk Classification: Low

Recommended Control: Apply data minimization and return only fields required by the client and permitted by the user's authorization.

Evidence 4 — GET /users: user-related fields returned



What the evidence demonstrates: The screenshot supports the observed request/response behavior described above. It is not, by itself, proof of an exploitable vulnerability.

5.5 Test 5 — /users?id=1 Query

Request: GET https://jsonplaceholder.typicode.com/users?id=1

Purpose: The test examined normal query-parameter handling, targeted resource retrieval, authentication requirements, and returned fields.

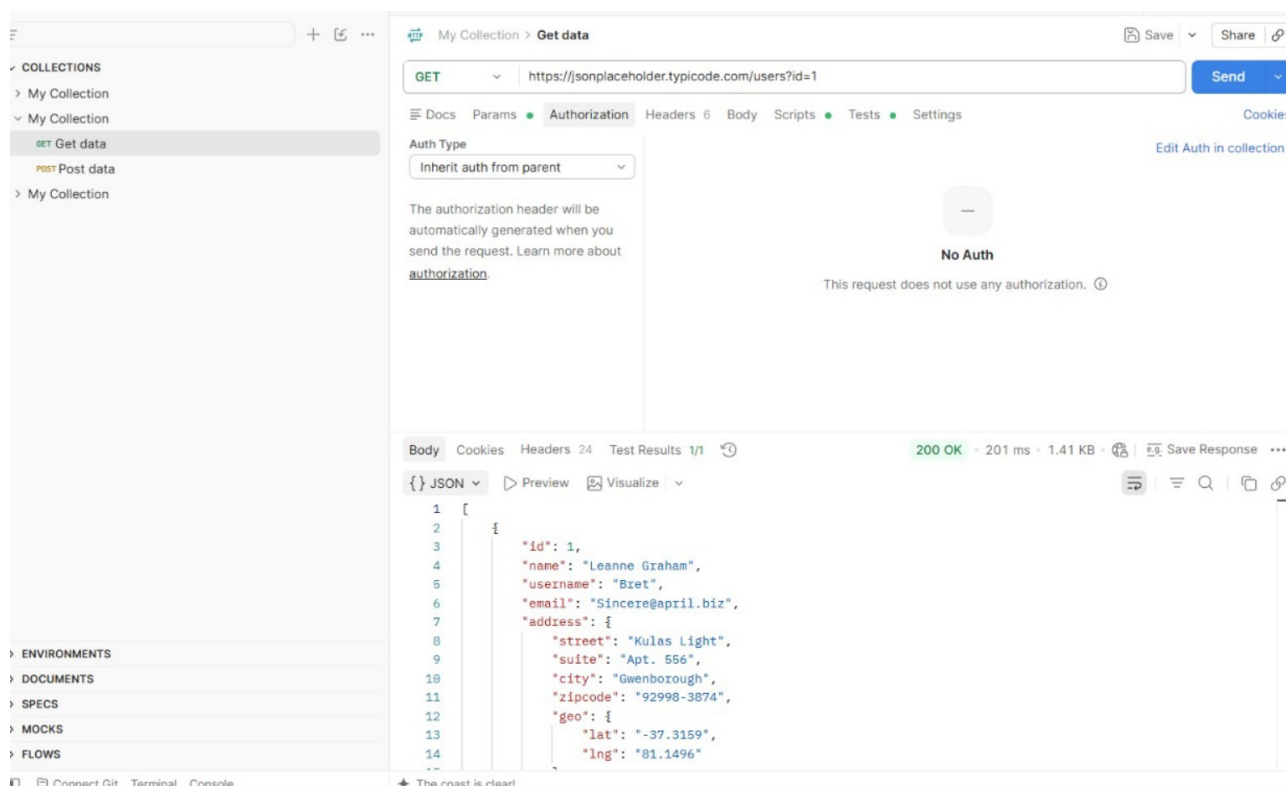
Observed Result: HTTP 200 OK returned the user corresponding to id=1 without authentication.

Security Analysis: In a private application, targeted user resources should be protected by authentication and authorization. No unauthorized/private account access was attempted.

Risk Classification: Low / Informational

Recommended Control: Validate query parameters, require authentication for private user resources, and enforce object-level authorization and data minimization.

Evidence 5 — GET /users?id=1: targeted user retrieval, No Auth, HTTP 200 OK



What the evidence demonstrates: The screenshot supports the observed request/response behavior described above. It is not, by itself, proof of an exploitable vulnerability.

6. Security Risk Summary & Severity Matrix

ID	Observation	Evidence	Severity	Confirmed vulnerability?
F-01	Unauthenticated access to /posts	E1	Low / Info	No
F-02	Direct retrieval of /posts/1	E3	Low / Info	No
F-03	Multiple user attributes via /users	E4	Low	No
F-04	Targeted user retrieval via /users?id=1	E5	Low / Info	No
F-05	Credentialed CORS setting observed	E2	Low / Info	No

6.1 Risk Interpretation

The assessment did not identify a confirmed exploitable vulnerability in the JSONPlaceholder test API. The observations are security considerations that would require stronger controls if the same API design were used to serve private or sensitive production data.

6.2 Overall Assessment

Overall Risk: LOW / INFORMATIONAL

The severity ratings are deliberately conservative and reflect the public/demo nature of the target, the use of synthetic data, the observed evidence, and the fact that no exploitation was performed.

7. Business Impact & Remediation Recommendations

7.1 Potential Business Impact in a Production SaaS API

Unauthorized data access: Protected resources exposed without authentication could lead to privacy violations, loss of trust, data leakage, and compliance consequences.

Broken object-level authorization: If a production API failed to verify resource ownership or permissions, users could potentially access other users' resources.

Excessive data exposure: Unnecessary personal or location information increases privacy risk and the amount of data available to unauthorized parties.

CORS misconfiguration: Overly permissive cross-origin policies can increase browser-based attack risk when sensitive authenticated data is involved.

7.2 Remediation Recommendations

Authentication: Require strong authentication for protected endpoints; use secure token/session management and appropriate unauthorized responses.

Authorization: Perform server-side object-level authorization checks for every protected resource and operation.

Data minimization: Return only the fields required by the client; avoid unnecessary personal, location, and internal fields.

CORS: Use a strict allowlist of trusted origins and enable credentialed cross-origin access only when necessary.

Rate limiting: Apply appropriate per-user, per-token, or per-IP limits to reduce automated abuse and resource exhaustion.

Input validation: Validate query parameters for type, range, format, and allowed values before processing.

Logging & monitoring: Monitor authentication failures, authorization denials, unusual request patterns, and sensitive endpoint access.

8. Conclusion

The API Security Risk Analysis was successfully conducted on the JSONPlaceholder public test API using Postman. The assessment followed a controlled, read-only methodology and focused on authentication, object-level access, response headers, data exposure, and query-parameter handling.

The testing showed that selected endpoints could be accessed without authentication, individual resources could be retrieved directly, user-related fields were returned, and a specific user could be queried using an ID parameter. Relevant response-header configuration was also observed.

These observations demonstrate important API security principles involving authentication, authorization, data minimization, CORS configuration, and secure API design.

However, JSONPlaceholder is intentionally designed as a public demonstration API and uses synthetic data. Therefore, the observed behavior was not classified as a confirmed exploitable vulnerability.

The exercise demonstrates that effective API security analysis requires more than identifying unusual behavior. Findings must be evaluated according to intended functionality, data sensitivity, authentication model, authorization model, and business context.

Final Assessment: LOW / INFORMATIONAL

No confirmed exploitable vulnerability was established within the defined read-only scope.

Key learning outcome: API security testing should identify and assess risk responsibly without unnecessarily exploiting or damaging the target system.

9. References & Evidence Index

9.1 References

1. JSONPlaceholder — Public REST API for testing and learning: <https://jsonplaceholder.typicode.com/>
2. JSONPlaceholder Guide — API resources and routes: <https://jsonplaceholder.typicode.com/guide/>
3. OWASP API Security Project — API security risks and guidance: <https://owasp.org/www-project-api-security/>
4. Postman — API testing and development platform: <https://www.postman.com/>

9.2 Evidence Index

Evidence 1 — GET /posts: public endpoint, No Auth, HTTP 200 OK.

Evidence 2 — Response headers inspected for GET /posts.

Evidence 3 — GET /posts/1: direct object retrieval, No Auth, HTTP 200 OK.

Evidence 4 — GET /users: user-related fields returned.

Evidence 5 — GET /users?id=1: targeted user retrieval, No Auth, HTTP 200 OK.

9.3 Ethical Statement

Testing was limited to the approved public/demo API and read-only requests. No exploitation, authentication bypass, denial-of-service testing, destructive requests, or attacks against private/production systems were performed.

— End of Report —